BeverageMart Frontend Task Breakdown

Three developers · Customer app (Expo, React Native) + Store owner dashboard (React) · one shared Django API · 5 weeks

How to read this document. Every task is small (a quarter of a day to two days). Each has an ID (for example **A-07**), the work, how we know it is done, the size in days, and the tasks it waits for. Each task is one card on the GitHub Projects board; use the ID in the branch name and pull request title, for example `feat/A-07-token-refresh`. **New to the project? Start with the "Start here" pages.** `docs/FRONTEND_HANDOFF.md` has every endpoint, error code and sample response.

Who	Owns	Total size
Developer A	Shared foundation, sign in / sign up / passwords, profile, addresses, notifications (both apps), store profile	about 18 days
Developer B	Customer shopping: home, store, product, cart, checkout, payment, order tracking, my orders	about 16.5 days
Developer C	Owner dashboard: layout, dashboard, orders, products, low stock, analytics, export	about 15.5 days

Start here: from clone to your first pull request

Everything you need is in this document and in the repository. You should not need a call to get started. When something is unclear, follow **"When you are stuck"** below.

1. Accounts and installs (once)

What	Notes
GitHub account	The project lead adds you to the repository. Accept the invitation e-mail from GitHub.
Git	git-scm.com. On Windows keep the default options (it also installs Git Bash).
Docker Desktop	docker.com/products/docker-desktop. On Windows accept "Use WSL 2" and restart the computer. Open it once and wait until it says **Engine running**.
Node.js 20 LTS	nodejs.org. Check with `node -v` (v20.x).
Editor	VS Code or Cursor, with the ESLint and Prettier extensions.
Expo Go (Developer A and B)	From the Play Store or App Store on your phone. Android Studio (emulator) is optional.

2. Clone and start the backend (one command)

```
git clone https://github.com/heismussa/Bevarage-Market-Place-App.git
cd Bevarage-Market-Place-App

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -File scripts\setup-dev.ps1

# macOS / Linux
bash scripts/setup-dev.sh
```

The script creates **your own** `.env` with random local passwords (nobody needs to send you secrets), starts Docker, creates the database, loads demo data, and prints your demo logins and the address your phone should use. The first run takes 5 to 10 minutes. It is safe to run again at any time.

3. Check that it works (5 minutes)

1. Open `http://localhost:8000/api/docs/`.
2. Open `POST /api/v1/auth/login/`, press **Try it out**, send the customer phone `+255712345002` and the customer password the script printed.
3. Copy the `access` value, press **Authorize** at the top, paste it.
4. Run `GET /api/v1/stores/`. You should see **ABC Drinks** and **Fresh Beverages**. Done.

4. First days

ID	Who	Task	Done when	Days
S-01	All	Steps 1 to 3 above.	Stores list works in the API docs.	0.5

S-02	All	Read `docs/FRONTEND_HANDOFF.md` sections 1 to 6 and the sections for your screens. Try your screens' endpoints in the API docs.	You know which endpoints your first five tasks call.	0.5
S-03	B, C	Until A-02 and A-03 are merged (about day 2), save the sample JSON for your first screens from the handoff into a `mocks/` folder in your branch.	Mock files ready.	0.25
S-04	B, C	When A announces "A-01 to A-03 merged" on the board: `git pull`, then `cd frontend` and `npm install`, and start your app.	Your app opens.	0.25

5. Your daily routine

```
# start of day
docker compose up -d              # Docker Desktop must be running
git checkout main
git pull
cd frontend
npm install                       # only if package.json changed
npm run gen:api                   # only if docs/openapi.yaml changed

# work on one task
git checkout -b feat/B-11-cart-screen
git add .
git commit -m "feat(customer): add cart screen"
git push -u origin feat/B-11-cart-screen
#   -> open the pull request on GitHub; the template lists what to check

# end of day
docker compose stop               # keeps your data
```

- One task = one branch = one pull request. Name: `feat/<task-id>-<short-name>` or `fix/...`.
- Another developer reviews and approves; then use **Squash and merge** and delete the branch.
- Review teammates' pull requests within one working day.

6. Picking tasks

- The tasks are cards on the GitHub **Projects** board, with columns **To do**, **In progress**, **In review** and **Done**.
- Take the next task in your list, assign yourself and move the card to In progress. Put the task ID in the pull request title. Move the card to Done when it is merged.
- Only start a task when everything in its **Waits for** column is Done, or build it against mock JSON and connect it later.

7. When you are stuck (no call needed)

1. Search `docs/FRONTEND_HANDOFF.md` (Ctrl+F the endpoint or the error code).
2. Try the same request in the API docs page.
3. Check the troubleshooting table below.
4. Still stuck: open a GitHub **Issue** with the **"Backend question or problem"** template (request, response, error code). Work on another task while you wait for the written answer.
5. Found a bug in a teammate's screen: open an Issue with the **"Bug in the app"** template and assign it to them.

Every Friday, each developer adds a short screen recording (1 to 2 minutes) of what they finished to their merged pull requests or the team chat.

8. Useful backend commands

Need	Command
See the forgot-password code	`docker compose logs backend` and look for "SMS to"
Approve a mobile-money payment	`docker compose exec backend python manage.py simulate_mobile_money <transaction_reference>` (from the `pay/` response)
Decline it	Same command with `--fail` at the end
Reset to clean demo data	`docker compose down -v`, then run the setup script again
Backend errors	`docker compose logs -f backend`
Your demo passwords	Open `.env` and look for `DEMO_..._PASSWORD`

9. Troubleshooting

Problem	Fix
"running scripts is disabled" on Windows	Use the exact command above with `-ExecutionPolicy Bypass`.
"Docker is not running"	Open Docker Desktop and wait for "Engine running", then try again.
Port 8000 or 5432 is already in use	Another program (often a local PostgreSQL) uses it. Stop that program or service, then run the script again.
Phone (Expo Go) cannot reach the API	Phone and computer on the same Wi-Fi. Use the "From your phone" address the script printed as `EXPO_PUBLIC_API_URL`. Allow port 8000 in Windows Firewall for private networks. Android emulator: use `http://10.0.2.2:8000/api/v1/`.
Browser says "blocked by CORS policy"	Run the dashboard on `http://localhost:5173` (Vite default). For another port, add it to `CORS_ALLOWED_ORIGINS` in `.env` and run `docker compose up -d`.
Demo login fails	Use the passwords in your own `.env`. If you changed them after the first run, reset demo data (see above).
`401 TOKEN_INVALID` on every call	The access token expired (30 minutes) or the password changed. Log in again; check the refresh logic (task A-07).
Payment screen waits forever	Mobile money is simulated: run `simulate_mobile_money` with the order's `transaction_reference`.
TypeScript errors right after `git pull`	Run `npm install` and `npm run gen:api` in `frontend/`.

Git asks for a password when pushing	Sign in with GitHub Desktop, or run `gh auth login`, or use a personal access token as the password.

10. Five rules

1. Never push to `main`; always a pull request.
2. Never commit `.env`, passwords, tokens or secrets.
3. Never change files in `backend/`; ask in an Issue instead.
4. Never calculate prices, totals or statuses in the app; show what the server sends.
5. Questions and decisions go in GitHub Issues, so everyone can read the answer later.

Developer A Foundation, accounts and notifications

A starts first. Tasks A-01 to A-12 should be merged by **day 3 of week 1**, because B and C use them. Until then, B and C build screens with the sample JSON from the handoff.

A1. Project foundation (week 1)

ID	Task	Done when	Days	Waits for
A-01	Create `frontend/` as an npm workspace with folders `packages/api`, `packages/ui-tokens`, `customer-app`, `owner-dashboard`.	`npm install` at `frontend/` works.	0.5	S-04
A-02	Create the Expo app (TypeScript, Expo Router, NativeWind).	Opens on a real phone with Expo Go.	0.5	A-01
A-03	Create the owner dashboard (Vite, React, TypeScript, React Router, Tailwind, shadcn/ui).	Opens at `localhost:5173`.	0.5	A-01
A-04	ESLint, Prettier and TypeScript strict mode for both apps; `npm run lint` and `npm run typecheck`.	Both commands pass.	0.5	A-02, A-03
A-05	Script `npm run gen:api` that generates `packages/api/src/schema.d.ts` from `docs/openapi.yaml`.	Types generate; a sample type compiles.	0.25	A-01
A-06	Typed API client (openapi-fetch). Base URL from `EXPO_PUBLIC_API_URL` / `VITE_API_URL`. Adds `Authorization: Bearer`. Add `.env.example` for each app.	`GET /stores/` works from both apps.	0.5	A-05
A-07	Token refresh: on `401 TOKEN_INVALID` call `auth/refresh/` once and retry; if refresh fails, log out. Only one refresh at a time when several calls fail together.	Expired access token is refreshed silently; a revoked one logs out.	1	A-06
A-08	Error helper: read `error.code`, `message`, `details`; put field errors on form fields (react-hook-form).	A failed sign-up shows the phone error under the phone input.	0.5	A-06
A-09	Formatters: money `"1500.00"` to `TSh 1,500`; dates in Africa/Dar_es_Salaam; phone input that sends `+255XXXXXXXXX`.	Unit tests pass.	0.5	A-01
A-10	Label maps for order status and payment status (handoff section 3, e.g. CANCELLED payment = "Not charged").	Exported from `packages/api`.	0.25	A-01
A-11	Theme tokens: dark (both apps) and light (dashboard) colours, radius, spacing, taken from the mockups.	Both apps use the tokens.	0.5	A-02, A-03

A-12	Auth state: save tokens (SecureStore on phone, localStorage on web), load on start, call `GET auth/me/`, send CUSTOMER to the app and STORE_OWNER to the dashboard.	Reopening the app keeps you logged in; wrong role sees "not supported".	1	A-07

A2. Customer app: accounts (week 1 to 2)

ID	Task	Done when	Days	Waits for
A-13	Splash screen (logo, then go to login or home).	Matches the mockup.	0.25	A-12
A-14	Log in screen (`POST auth/login/`). Show "wrong phone or password" for `AUTHENTICATION_FAILED`.	Demo customer can log in.	0.5	A-12
A-15	Sign up screen (`POST auth/register/`, role CUSTOMER), then log in automatically.	New user lands on Home.	0.5	A-14
A-16	Forgot password, step 1: enter phone (`POST auth/password/reset/`).	Shows the server message; goes to step 2.	0.25	A-14
A-17	Forgot password, step 2: code + new password (`auth/password/reset/confirm/`). Handle wrong code, weak password, `THROTTLED`.	Reset works with the code from `docker compose logs backend`.	0.5	A-16
A-18	Profile screen: show and edit name, phone, email (`GET/PATCH auth/me/`).	Changes are saved.	0.5	A-12
A-19	Change password (`auth/password/change/`); save the new tokens it returns.	Still logged in here; other device is logged out.	0.5	A-18
A-20	Log out button (delete tokens, go to login).	Works.	0.1	A-18
A-21	Addresses list: show, delete, set default (`addresses/`, `set-default/`).	Default address is marked.	0.5	A-12
A-22	Add / edit address form with "use my location" (sends `latitude` / `longitude` as strings).	New address appears in the list.	1	A-21

A3. Notifications, both apps (week 3)

ID	Task	Done when	Days	Waits for
A-23	Bell icon with unread count (`notifications/unread-count/`), polled every 30 to 60 s, paused when the app is hidden.	Count goes up after placing an order.	0.5	A-12
A-24	Customer notification list: pagination, mark one read, mark all read, tap opens the order.	Read items look different.	1	A-23
A-25	Owner top bar: bell, notification dropdown, profile menu, dark / light switch.	Works on every dashboard page.	1	A-23, C-01

A4. Owner dashboard: accounts and store profile (week 2 to 3)

ID	Task	Done when	Days	Waits for
A-26	Owner log in and sign up pages (role STORE_OWNER).	Demo owner can log in.	0.5	A-12
A-27	Owner forgot password and change password pages (reuse the customer logic).	Both flows work on the web.	0.5	A-26
A-28	First store: an owner with no store is sent to a "create your store" form (`POST owner/stores/`).	New owner can create a store.	1	A-26
A-29	Store profile page: edit details and upload a logo (JPEG / PNG / WebP under 2 MB, compress first).	Logo shows in the customer app.	1	A-28
A-30	"Accept new orders" switch mapped to store `status` OPEN / CLOSED.	Closed store shows "Closed" to customers.	0.25	A-29
A-31	Review pass: loading, empty and error states on every screen of both apps.	Checklist signed off with B and C.	1	week 4

Developer B Customer shopping and ordering

Week 1: build the screens with the sample JSON from the handoff. Connect to the API once A-06 and A-12 are merged.

B1. App frame and shared pieces (week 1)

ID	Task	Done when	Days	Waits for
B-01	Bottom tabs (Home, Orders, Cart, Profile) and screen stack.	Tabs switch; matches the mockup.	0.5	A-02
B-02	Shared components: button, input, store card, product card, badge, quantity stepper, loading skeleton, empty state, error with "Try again".	Used by at least two screens.	1.5	A-11

B2. Browse (week 1 to 2)

ID	Task	Done when	Days	Waits for
B-03	Home: ask for location, show the location chip, list stores near the user (`stores/?lat=&lng=&ordering=distance`); the chip can also filter by `city` .	Distance shows on each card.	1	B-02, A-06
B-04	Home: search bar (searches stores) and category chips (`categories/` , `stores/?category=`).	Filtering changes the list.	0.5	B-03
B-05	Home: infinite scroll, pull to refresh, "Open now" / "Closed" badge.	Works with many stores.	0.5	B-03
B-23	Home: "Popular products" row (`products/popular/?page_size=10`) showing image, name, price and an add button. Hide the whole row when the list is empty.	Products sold in the demo appear, best seller first.	0.5	B-03
B-06	Store page header: logo, name, open / closed, delivery fee, distance (`stores/{id}/`).	Matches the mockup.	0.5	B-03
B-07	Store products grid with search, category filter and infinite scroll (`stores/{id}/products/`).	Search finds "Coca".	1	B-06
B-08	Product detail: image, price, stock, availability, quantity stepper (`products/{id}/`).	Out-of-stock product cannot be added.	0.5	B-07

B3. Cart and checkout (week 2 to 3)

ID	Task	Done when	Days	Waits for
B-09	Add to cart from card and detail (`POST cart/items/`); cart badge count on the tab.	Badge updates.	0.5	B-08

B-10	"Start a new cart?" dialog for `CART_STORE_CONFLICT` (items from another store).	Choosing yes empties and adds.	0.5	B-09
B-11	Cart screen: lines, change quantity, remove, show `warnings`, subtotal from the server. Replace cart state with every response.	Numbers always match the server.	1	B-09
B-12	Checkout: choose delivery address; link to "add address".	Default address is preselected.	0.5	B-11, A-21
B-13	Checkout: payment choice (Cash, M-Pesa, Tigo Pesa, Airtel Money) and notes.	No card option shown.	0.5	B-12
B-14	Place order (`POST orders/`): button disabled while sending; handle `STORE_CLOSED`, `INSUFFICIENT_STOCK`, `PRODUCT_UNAVAILABLE`, `EMPTY_CART`. Never retry automatically.	Cash order opens the tracking screen.	1	B-13

B4. Payment and orders (week 3 to 4)

ID	Task	Done when	Days	Waits for
B-15	Mobile money: call `orders/{id}/pay/`, show "Approve the payment on your phone".	Screen shows after placing an M-Pesa order.	0.5	B-14
B-16	Poll `orders/{id}/payment/` every 3 s for up to 2 minutes. Show Paid, Failed with "Try again", or "Still waiting".	Works with `simulate_mobile_money` and `--fail`.	1	B-15
B-17	Order tracking: status timeline (`status_history`), items, totals, payment label.	Matches the mockup.	1	B-14
B-18	Tracking refresh every 15 to 30 s; stop when the order is finished or the screen is hidden.	Owner's changes appear on the phone.	0.5	B-17
B-19	Cancel order while PENDING, with a reason (`orders/{id}/cancel/`). Button only when `allowed_actions` allows it.	Cancelled order shows "Not charged".	0.5	B-17
B-20	My orders: list, status filter, infinite scroll, open detail.	Newest first.	1	B-17
B-21	Refund notice: a paid order that is cancelled or rejected shows "Contact the store about your refund".	Shows only in that case.	0.25	B-17
B-22	Edge case test pass: closed store, sold out, lost network while placing an order, expired login.	No crash, clear message for each.	1	week 4

Developer C Store owner dashboard

Week 1: layout and static pages with the sample JSON. Connect to the API once A-06 and A-12 are merged. Every page works in dark and light themes.

C1. Frame and shared pieces (week 1)

ID	Task	Done when	Days	Waits for
C-01	Layout: sidebar (Dashboard, Orders, Products, Low stock, Sales Analytics, Store Profile), top bar area, content area; works on laptop and tablet.	Matches the mockup.	1	A-03
C-02	Shared components: data table, pagination, status badge, date range picker, confirm dialog with reason, toast messages.	Used by orders and products.	1.5	A-11
C-03	Store switcher (`owner/stores/`); remember the chosen store.	Pages reload for the chosen store.	0.5	C-01, A-12

C2. Dashboard (week 1 to 2)

ID	Task	Done when	Days	Waits for
C-04	Number cards (`owner/stores/{id}/dashboard/`): orders and sales of the last 7 days from `last_7_days_vs_previous.current` with a "+12.5% vs last week" badge from `orders_change_percent` / `sales_change_percent` (green up, red down, "New" when `null`); pending orders; low-stock count.	Numbers and badges match the API.	0.5	C-03
C-05	Sales chart for the last 7 days (Recharts, from `sales_last_7_days`).	Matches the mockup.	0.5	C-04
C-06	Recent orders table and low-stock list on the dashboard.	Click opens order / product.	0.5	C-04
C-22	"Payment methods" donut chart from `payment_methods_last_7_days` (labels M-Pesa, Tigo Pesa, Airtel Money, Cash; percentages from `orders_percent`). Empty state when all are 0. Reuse the component on the analytics page with `payment_methods` .	Matches the mockup on both pages.	0.5	C-04
C-07	Refresh pending orders every 15 to 30 s; highlight new ones.	New customer order appears without reload.	0.5	C-04

C3. Orders (week 2)

ID	Task	Done when	Days	Waits for

C-08	Orders list (`owner/stores/{id}/orders/`): status tabs, search by order number or customer, date range, sort, pagination.	All filters work together.	1.5	C-02, C-03
C-09	Order detail (`owner/orders/{id}/`): customer, address, items, totals, payment label, history timeline.	Matches the mockup.	1	C-08
C-10	Action buttons from `allowed_actions` only (`owner/orders/{id}/transition/`). Reason dialog for reject and cancel. On `INVALID_TRANSITION` reload the order.	Accept, prepare, ready, complete all work.	1	C-09

C4. Products and low stock (week 3)

ID	Task	Done when	Days	Waits for
C-11	Products table (`owner/stores/{id}/products/`): image, name, price, stock, availability, search, category filter, pagination.	Matches the mockup.	1	C-02
C-12	Add product form with image upload (under 2 MB, compressed), category, price, stock, low-stock threshold, availability.	New product shows in the customer app.	1	C-11
C-13	Edit product form (`PATCH owner/products/{id}/`).	Changes saved.	0.5	C-12
C-14	Quick stock update in the table (`owner/products/{id}/stock/`).	Stock changes without opening the form.	0.5	C-11
C-15	Delete product with a confirm dialog.	Product disappears from both apps.	0.25	C-11
C-16	Low stock page (`?low_stock=true`) with a quick restock button.	Lists products at or below the alert level.	0.5	C-14

C5. Sales analytics and extras (week 4)

ID	Task	Done when	Days	Waits for
C-17	Analytics page: date range picker and total cards (`owner/stores/{id}/analytics/`).	Changing dates updates the cards.	0.5	C-02
C-18	Daily sales line chart and top products bar chart.	Matches the mockup.	1	C-17
C-19	"vs previous period" on analytics cards from `compared_to_previous`. For "This week vs last week", send Monday to today as the range.	Percentage shows with up / down arrow.	0.25	C-17

C-20	Orders "Export" button: fetch `owner/stores/{id}/orders/export/` with the current filters and the Bearer token, save the response as `orders.csv` (handoff shows the code). On `VALIDATION_ERROR` ask for a shorter date range.	File opens in Excel with the filtered orders.	0.5	C-08
C-21	Light theme check on every page.	No unreadable text in either theme.	0.5	A-25

Everyone Week 5: finish and demo

ID	Task	Done when	Days
T-01	Run the full demo script below on fresh seeded data, all three together.	No step fails.	0.5
T-02	Bug fixing from the demo run and from each other's testing.	No open "must fix" bugs.	2
T-03	Build the Android APK (Expo EAS build) and install it on two phones.	APK runs against the server URL.	0.5
T-04	Production build of the dashboard (`npm run build`).	Build folder serves correctly.	0.25
T-05	A short README for each app: how to install, run, and set the API URL.	A new teammate can run it alone.	0.5

Demo script (what the finished product must do)

1. Owner signs up on the dashboard, creates a store, uploads a logo, adds products.
2. Customer signs up on the phone, allows location, sees the store on Home, opens it, adds two products, checks out with M-Pesa.
3. Run `simulate_mobile_money`; the payment screen changes to "Paid".
4. The owner's bell shows the new order; the pending count goes up.
5. Owner accepts, prepares, marks ready and completes. The customer's tracking screen follows each step and the bell shows each update.
6. Stock goes down; a low-stock alert appears when a product reaches its alert level.
7. Dashboard and analytics show the sale, the "vs last week" badge and the payment methods chart; the owner exports the orders to CSV.
8. The sold products appear under "Popular products" on the customer's Home.
9. Customer uses "Forgot password"; the code is read from the backend log.

Week-by-week view

Week	A	B	C
1	A-01 to A-15	B-01, B-02, B-03 to B-08 with sample JSON	C-01 to C-07, C-22 with sample JSON
2	A-16 to A-22, A-26, A-27	Connect B-03 to B-08 and B-23; B-09 to B-11	C-04 to C-07, C-22 connected; C-08 to C-10
3	A-23 to A-25, A-28 to A-30	B-12 to B-17	C-11 to C-16
4	A-31; help B and C	B-18 to B-22	C-17 to C-21
5	T-01 to T-05		

Team rules

- **Git:** nobody pushes to `main`. One branch per task (`feat/B-11-cart-screen`), a small pull request, one review by a teammate, then merge.
- **Commits:** short and clear, for example `feat(customer): add cart screen`.
- **Secrets:** never commit `.env`, demo passwords, tokens or the payment secret. App env files hold only the API URL.
- **Shared code:** only A changes `packages/api` and `packages/ui-tokens`; others ask A or open a small pull request.
- **Backend:** nobody edits the backend. Send questions to the project lead with the request, the response and the `error.code`.
- **Every morning:** pull `main`; if `docs/openapi.yaml` changed, run `npm run gen:api`.
- **Meetings:** 15-minute check-in every day; demo on seeded data every Friday.

Definition of done (every task)

- Uses the generated API types; no hand-written response types.
- Has loading, empty, error and success states.
- Handles the error codes the endpoint can return; field errors appear under the right input.
- Never calculates prices, totals or statuses; shows the values the server sends.
- Buttons that call the API are disabled while the request is running.
- Checked against the mockup and the "must / must not" list in the handoff.
- `npm run lint` and `npm run typecheck` pass; reviewed by a teammate.

Mockup items: built or hidden

Built in the backend for this version: popular products (B-23), "+12% vs last week" (C-04), payment methods chart (C-22), "this week vs last week" on analytics (C-19), orders CSV export (C-20).

Hidden in this version (they need database changes and come in a later version). Do not show fake data.

Mockup item	What to do for now
Store rating "4.5 (120)", favourite heart	Hide.
Opening hours	Show "Open now" / "Closed" from the store status.
Notifications / low-stock switches in settings	Hide; only "Accept new orders" (OPEN / CLOSED) works.
Card payment	Hide; offer Cash, M-Pesa, Tigo Pesa, Airtel Money.
One search for orders, products and customers	Search inside the current page's list.